Lab 6: Design sequential circuit (cnt)

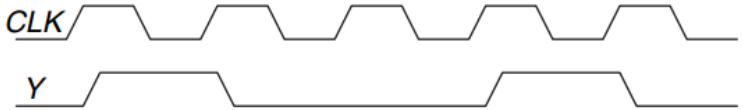
Finite state machine

Finite state machine consists of:

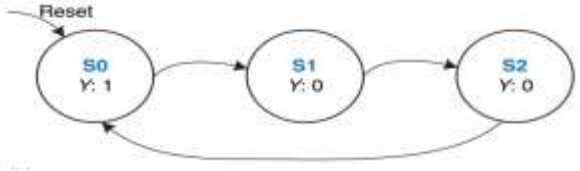
- **Two blocks of combinational circuit**: next state logic and output logic
  + **Next state logic:** determine next state base on current state and current inputs.
  + **Output logic:**
    Moore machines: outputs depend only on current state of machine.
    Mealy machines: outputs depend on both current state of machine and current inputs.

- **Registers** that store states.

Finite state machine



Divide by 3 counter waveform



Divide by 3 counter state transition diagram

Current State	Next State
S0	S1
S1	S2
S2	S0

State transition table

Current State	Output
S0	1
S1	0
S2	0

Output table

Finite state machine

divideby3FSM.sv

```
module divideby3FSM(input logic clk,
      input logic reset,
      output logic y);

                  typedef enum logic [1:0] {S0, S1, S2} statetype;
                  statetype state, nextstate;

                  // state register
                  always_ff @(posedge clk, posedge reset)
                  if (reset) state <= S0;
                  else state <= nextstate;

                  // next state logic
                  always_comb
                  case (state)
                  S0: nextstate = S1;
                  S1: nextstate = S2;
                  S2: nextstate = S0;
                  default: nextstate = S0;
                  endcase

                  // output logic
                  assign y = (state == S0);
endmodule
```

divideby3FSM_tb.sv

```
module divideby3FSM_tb();
  logic clk;     // Clock signal
  logic reset;   // Reset signal
  logic y;      // Output signal

  // Instantiate the module under test
  divideby3FSM DUT(clk, reset, y);

  // Clock generation
  always begin
    #5 clk = ~clk; // Toggle the clock every 5 time units
  end

  // Initialize signals
  initial begin
    clk = 0;
    reset = 1;
    #10 reset = 0; // Release reset

    // Test sequence
    $display("Time   State   Nextstate   Output");
    for (int i = 0; i < 15; i = i + 1) begin
      #5; // Advance time

      // Display the current state and output
      $display("%0d    %0s    %0s    %b", $time, DUT.state, DUT.nextstate, y);
    end

    $stop; // End simulation
  end
endmodule
```

Syntax

enum is a user-defined data type used to giving meaningful names to constant values. enums are often used to improve code readability and maintainability.

Syntax:

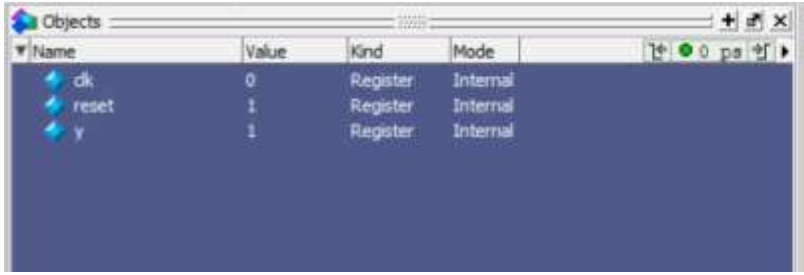
```
typedef enum datatype [bitwidth:0] {
   enumerator1 = constant1,
   enumerator2 = constant2,
   // More enumerators...
} enum_type;
```

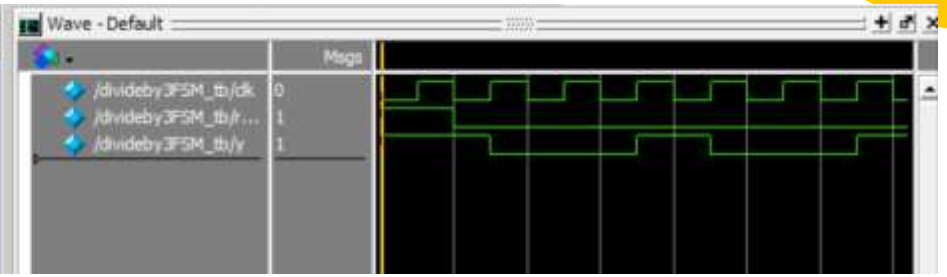
$display is a system function is used for displaying text messages and variable values in a simulation environment.

Syntax:

```
$display("format string", expression1, expression2, ...);
```

Simulation result



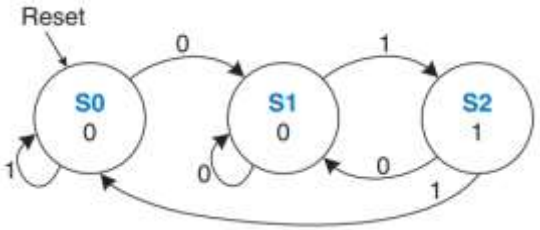


Transcript

```
# .main_pane.structure.interior.cs.body.struct
# view signals
# .main_pane.objects.interior.cs.body.tree
# run -all
# Time     State     Nextstate     Output
# 15       S0        S1            1
# 20       S1        S2            0
# 25       S1        S2            0
# 30       S2        S0            0
# 35       S2        S0            0
# 40       S0        S1            1
# 45       S0        S1            1
```

Exercise 1

Design a digital circuit based on following Moore state transition diagram using SystemVerilog and simulate on ModelSim.



Moore machine state transition diagram

Current State S	Input A	Next State S'
S0	0	S1
S0	1	S0
S1	0	S1
S1	1	S2
S2	0	S1
S2	1	S0

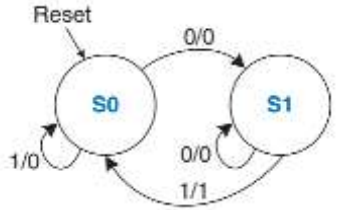
Moore state transition table

Current State S	Output Y
S0	0
S1	0
S2	1

Output table

Exercise 2

Design a circuit based on following Mealy state transition diagram using SystemVerilog and simulate on ModelSim.



Mealy machine state transition

Current State S	Input A	Next State S'	Output Y
S0	0	S1	0
S0	1	S0	0
S1	0	S1	0
S1	1	S0	1

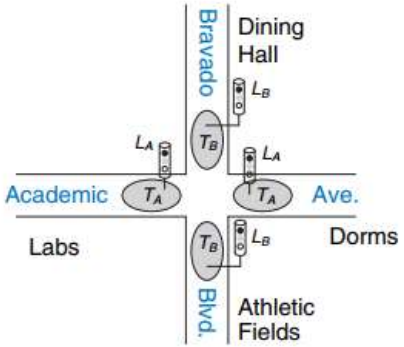
Mealy state transition and output table

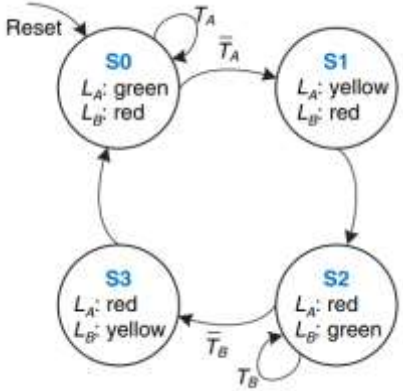
EX3. Traffic light controller

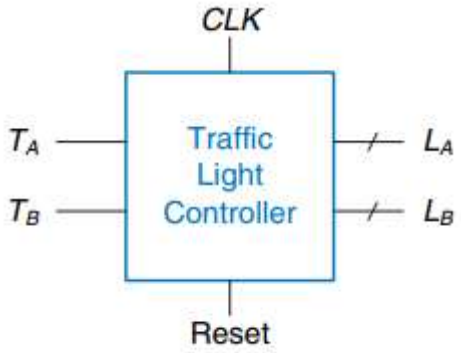
Students move from Labs to Dorms on Academic street.
Football players move from Dining hall to Athletic Fields on Bravado street.

Input: Two traffic sensors, TA and TB. TA indicates true if students are present and false if Academic street is empty. TB indicates true if football player are present and false if Bravado street is empty.

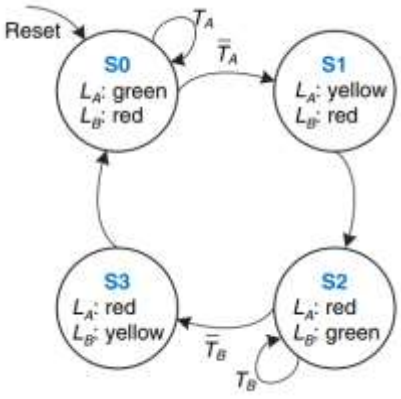
Output: Two traffic lights LA and LB to control traffic. Each lights receives digital inputs to specify whether it should be yellow, red, green.







Design a light traffic controller digital circuit based on following state transition diagram using SystemVerilog and simulate on ModelSim.



Current State S	Inputs T_A	Inputs T_B	Next State S'
S0	0	X	S1
S0	1	X	S0
S1	X	X	S2
S2	X	0	S3
S2	X	1	S2
S3	X	X	S0

State transition table

Output	Encoding $L_{1:0}$
green	00
yellow	01
red	10

Output encoding

State	Output
S0	LA: green, LB: red
S1	LA: yellow, LB: red
S2	LA: red, LB: green
S3	LA: red, LB: yellow

Output table